Vidyavardhini's College of Engineering and Technology, Vasai

Department of Computer Science & Engineering (Data Science)

EXPERIMENT ASSESSMENT

ACADEMIC YEAR 2025 - 26

Course: Machine Learning Lab

Course code: CSL604

Year: TE SEM: VI

Experiment Number: 08
Aim: To implement Error Backpropagation Perceptron Training Algorithm.
Name: Satyam Maurya
Roll Number: 62
Date of Performance: 16/3
Date of Submission: 30/3

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal Work & Timely Submission	10	
Total	20	

Performance Indicator	Expected Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	5	3	2
Understanding	5	3	2
Journal Work & Timely Submission	10	6	4

Checked by:

Name of Faculty: Ms. Nivedha Raut

Signature:

Date:

Aim: To implement Error Backpropagation Perceptron Training Algorithm.

Description:

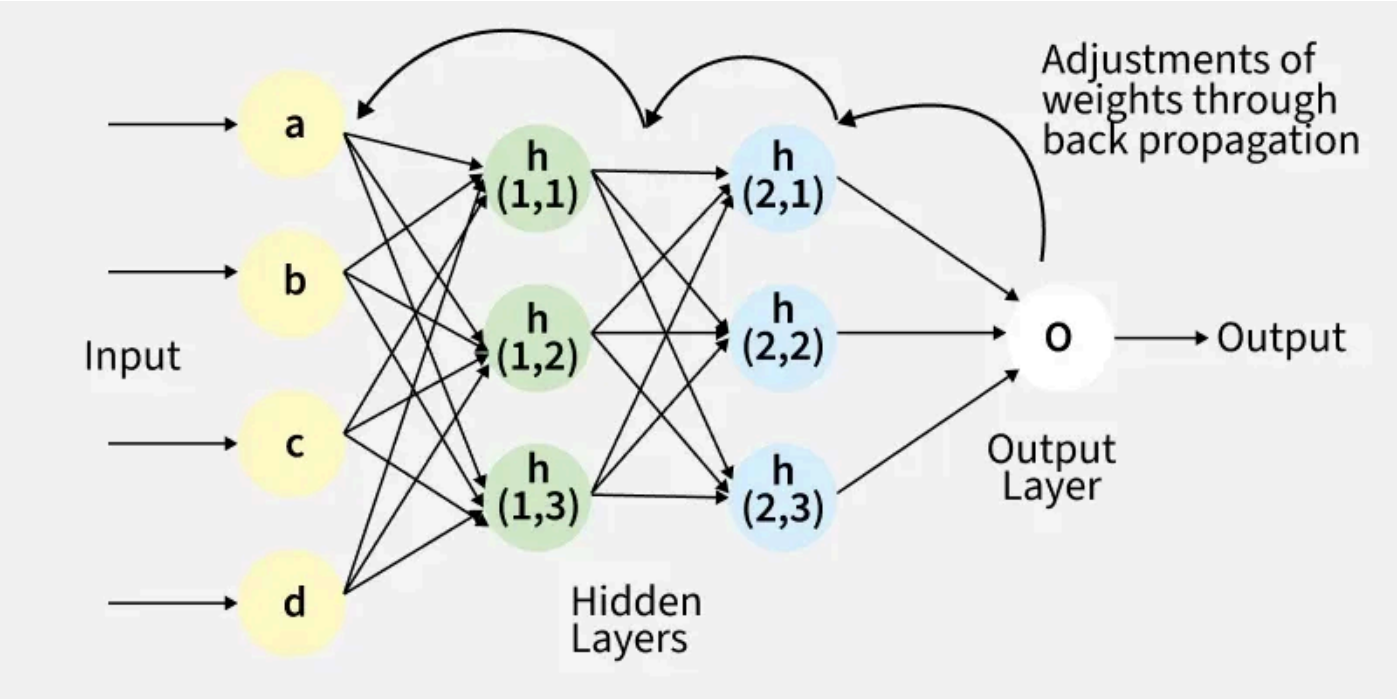
Error Backpropagation is a supervised learning algorithm used to train multilayer perceptron (MLP) networks. It is one of the most important algorithms in neural networks and deep learning.

The main goal of the backpropagation algorithm is to minimize the error between the predicted output and the actual output by adjusting the weights and biases of the network. It works on the principle of gradient descent optimization.

The algorithm operates in two main phases:

1. **Forward Pass:**
   The input data is passed through the network layer by layer to generate the output. Each neuron calculates a weighted sum of inputs and applies an activation function.
2. **Backward Pass:**
   The error is calculated as the difference between actual and predicted output. This error is propagated backward through the network, and weights are updated using derivatives (gradients) to reduce the error.

Backpropagation allows neural networks to learn complex non-linear relationships and is widely used in classification, regression, and pattern recognition problems.



Code:

```
import numpy as np

# Sigmoid function
def sigmoid(x):
    return 1/(1+np.exp(-x))

def derivative(x):
    return x*(1-x)

# Number of patterns
n = int(input("Enter number of patterns: "))

X = []
y = []

# Taking input from user
for i in range(n):
    print(f"\nPattern {i+1}")
    x1 = int(input("Enter x1: "))
    x2 = int(input("Enter x2: "))
    target = int(input("Enter target (0 or 1): "))

    X.append([x1, x2])
    y.append([target])

X = np.array(X)
y = np.array(y)

# Initialize weights
np.random.seed(1)
w1 = np.random.rand(2,2)
w2 = np.random.rand(2,1)

# Training
for i in range(10000):
    # Forward propagation
    hidden = sigmoid(np.dot(X, w1))
    output = sigmoid(np.dot(hidden, w2))

    # Error
    error = y - output
```

```
    # Backpropagation
    d_output = error * derivative(output)
    d_hidden = d_output.dot(w2.T) * derivative(hidden)

    # Update weights
    w2 += hidden.T.dot(d_output)
    w1 += X.T.dot(d_hidden)

print("\nFinal Output:")
print(output)
```

Output:

```
Enter number of patterns: 4

Pattern 1
Enter x1: 0
Enter x2: 0
Enter target (0 or 1): 0

Pattern 2
Enter x1: 0
Enter x2: 1
Enter target (0 or 1): 1

Pattern 3
Enter x1: 1
Enter x2: 0
Enter target (0 or 1): 1

Pattern 4
Enter x1: 1
Enter x2: 1
Enter target (0 or 1): 0

Final Output:
[[0.03295304]
 [0.93052381]
 [0.93052373]
 [0.09263959]]
```

Conclusion:

Q1. What is back propagation, and how does it work?

Backpropagation is a supervised learning algorithm used to train multi-layer neural networks by minimizing the error between actual and predicted outputs. It works by propagating the error backward through the network and updating the weights using gradient descent.

Q2. Why is backpropagation important in neural networks?

Backpropagation is important because it enables neural networks to learn complex non-linear relationships. It improves model accuracy by continuously adjusting weights and is widely used in deep learning applications.